

you

良心送分题。

30pts

暴力。

$l = r$ 部分分

由于我们只要求最后矩阵中的数字和，所以操作顺序与答案无关。

我们将操作进行排序，先处理行操作，然后处理出每一列的数字和 f_i ，那么列操作就是在 f_i 上的一维问题。

考虑如何算 f_i 。

我们可以先将每一行乘的数 mul_i 算出来，然后计算 f_i ，但是朴素的算法仍然是 $O(nm)$ 的，考虑优化这个暴力。

在原矩阵中，后一个数等于前一个数 +1，根据乘法分配律， $f_i = f_{i-1} + \sum_{i=1}^n mul_i$ ，于是可以线性递推。

正解

在上述 $l = r$ 的部分分做法中，我们已经把原问题转化为了一维问题，操作也转化成了区间乘。

那么我们定义差分数组 R_i 表示 $\frac{mul_i}{mul_{i-1}}$ ，那么一次操作就相当于在 R_l 处乘上 v ， R_{r+1} 处乘上 $\frac{1}{v}$ （本题保证 v 有逆元）

在最后利用 R_i 求出 mul_i ，使用上文所述的解法即可。

are

有一个显然的结论：我们只会以 1 号点为起点加边。

对于图是一棵内向树的情况，我们发现必然会存在一条从 1 到叶子结点的边，那么我们把这条边连上，并删除已经满足 $dis(1, i) \leq K$ 的点。此时整棵树会产生新的叶子结点，重复上述过程直到满足题目条件即可。

对于图是基环树的情况，可以把环提出来，然后对树执行上面的算法，执行完之后的局面是这样的：环上有一些点已经满足条件，一些点没有满足条件；不在环上的点全部满足条件。

考虑怎么解决环的情况，可以枚举起点 st ，连接一条 $(1, st)$ 的边并覆盖 $[st, st + k - 1]$ 这一整个区间，执行完之后，找到下一个不合法的点，并执行上述操作直到所有点都被覆盖，时间复杂度是 $O(\frac{n}{k} \cdot n)$ 的。

假设我们将每次覆盖的起点称作“断点”，可以发现一条性质：对于任意相邻的未被覆盖的 k 个点，必然有一个会成为断点。证明只需要考虑如果没有断点，则原方案中存在一次覆盖使得它覆盖了所有 k 个点，而这显然是不成立的。

所以找到相邻的未被覆盖的 k 个点，在这些点中枚举起点并尝试覆盖，复杂度是 $O(k \cdot \frac{n}{k}) = O(n)$ 的，可以通过本题。

测试点 1 ~ 4

一个 k 能给我们带来 $\sum_{i=1}^k a_i, \sum_{i=k+1}^{2k} a_i, \dots, \sum_{i=tk}^n a_i$ 的值 (其中 $t = \lfloor \frac{n}{k} \rfloor$)

记 $s_i = \sum_{x=1}^i a_x$

一个 k 就能告诉我们 $s_k - s_0, s_{2k} - s_k, \dots, s_n - s_{tk}$

也就是说, 我们根据一个 k , 可以知道 $s_k, s_{2k}, \dots, s_{tk}, s_n$ 的值

为方便起见, 接下来我们令 $k_{m+1} = n$, 并使 $m \leftarrow m + 1$

那么一个 a_i 已知, 当且仅当 s_i 与 s_{i-1} 均已知。

转化一下, 就是 $\exists j_1, j_2 \in [1, m], k_{j_1} | i, k_{j_2} | (i - 1)$

直接对于每个 i , 处理出是否存在一个 k_j 满足 $k_j | i$ 即可。

时间复杂度 $\Theta(nm)$ 。

测试点 7 ~ 8

这两个点满足 $m = 2$

那么上文中的 k_{j_1}, k_{j_2} 都是唯一的, 直接计算即可。

如果出现一个 k 同时整除 i 与 $i - 1$, 说明 $k = 1$, 特判即可。

测试点 11 ~ 16

这六个点满足 $m \leq 10$

我们为什么不能直接枚举 k_{j_1}, k_{j_2} 来计算, 因为有算重的可能, 即存在多对 j_1, j_2 , 都满足 $k_{j_1} | i, k_{j_2} | (i - 1)$, 但我们只能对于这个 i 计算一次。

设 $f(S, T)$ 表示满足 $j \in S \Leftrightarrow k_j | i, j \in T \Leftrightarrow k_j | (i - 1)$ 的 i 的个数, 也就是说整除 i 的 k 的集合恰好为 S , 整除 j 的 k 的集合恰好为 T 的方案数。

那么 $Ans = \sum_{S \neq \emptyset, T \neq \emptyset} f(S, T)$

我们设 $A(S, T)$ 表示满足 $j \in S \Rightarrow k_j | i, j \in T \Rightarrow k_j | i - 1$ 的 i 的个数, 注意这里的定义与上文不同, 这里我们只保证了 S 集合里的 k 都整除 i , 但未保证“恰好”。

$A(S, T)$ 是很容易求的, 因为原定义等价于 $\text{lcm}(k_j)_{j \in S} | i, \text{lcm}(k_j)_{j \in T} | (i - 1)$, 设 $a = \text{lcm}(k_j)_{j \in S}, b = \text{lcm}(k_j)_{j \in T}$, 则有 $ax - by = 1$, 使用拓展欧几里得算法计算出一组可能的 x, y , 然后利用二元一次不定方程的通解形式计算在 $[1, n]$ 中的 ax 有多少个即可。

但 $f(S, T)$ 难以直接计算, 因为“恰好”难以处理。

考虑子集反演, 建立 $f(S, T)$ 与 $A(S, T)$ 的关系。

为了辅助推导, 设 $g(S, T)$ 表示 $j \in S \Leftrightarrow k_j | i, j \in T \Rightarrow k_j | i - 1$ 的 i 的个数。

那么

$$\begin{aligned} g(S, T) &= \sum_{T \subseteq V} f(S, V) \\ \Rightarrow f(S, T) &= \sum_{T \subseteq V} (-1)^{|V| - |T|} g(S, V) \end{aligned}$$

那么

$$\begin{aligned}\sum_{T \neq \emptyset} f(S, T) &= g(S, \emptyset) - f(S, \emptyset) \\ &= \sum_{T \neq \emptyset} (-1)^{|T|-1} g(S, T)\end{aligned}$$

同理可得

$$\sum_{S \neq \emptyset} g(S, T) = \sum_{S \neq \emptyset} (-1)^{|S|-1} A(S, T)$$

那么有

$$\begin{aligned}\sum_{S \neq \emptyset, T \neq \emptyset} f(S, T) &= \sum_{S \neq \emptyset, T \neq \emptyset} (-1)^{|T|-1} g(S, T) \\ &= \sum_{T \neq \emptyset} (-1)^{|T|-1} \sum_{S \neq \emptyset} g(S, T) \\ &= \sum_{T \neq \emptyset} (-1)^{|T|-1} \sum_{S \neq \emptyset} (-1)^{|S|-1} A(S, T) \\ &= \sum_{S \neq \emptyset, T \neq \emptyset} (-1)^{|S|+|T|} A(S, T)\end{aligned}$$

枚举 S 与 T 直接计算上式，时间复杂度为 $\Theta(4^m \log n)$ 。

正解

我们之前提到，除了 $k = 1$ 的情况，不会有那个 k 同时整除 i 与 $i - 1$ 。

所以排除特殊情况后，我们可以发现， $A(S, T) \neq 0$ 可以推出 $S \cap T \neq \emptyset$

故只需枚举与 S 不交的集合 T 计算即可，时间复杂度为 $\Theta(3^m \log n)$ 。

strong

暴力做法

容易发现一条性质：每个人最多只会被移动一次。说明人只有两种：移动的和不移动的。

考虑枚举所有不移动的人，并最优化其它人的移动顺序。最开始第 i 个人的起点为 i ，终点为 p_i ，经过操作之后第 i 个人的起点和终点位置都有可能变化，我们希望操作之后起点终点编号尽可能小。可以想到一个贪心：每次选取起点最小的点并进行操作，证明可以考虑邻项交换。这样我们得到了一个 $\mathcal{O}(n2^n)$ 的做法。一个与之对称的方法是每次选取终点最大的点，下面会提到它的用处。

状态设计和贡献组成

尝试用动态规划优化掉指数。从前往后扫不太可行，考虑以值域为阶段。此时上面提到的第二种贪心就派上用场了，因为终点位置和值域有关，所以依照上面的贪心，可以考虑从 n 到 1 倒序排序。

定义 $f_{i,j}$ 表示倒序考虑到 i ，位置最小的不动点位置为 j 的最优解。在讨论具体转移之前，我们先讨论经过若干次操作之后，权值为 i 的人的实际位置。一个事实是：所有没有被操作的人的相对顺序是不变的，这部分的贡献是权值为 i 的人的位置前面所有未被操作的点。另一部分的贡献是位置在 i 之前的所有不动点的前面的连续递增子段的长度之和（因为我们在排序过程中会将与不动点值域连续的所有动点接在不动点的前面）。

费用提前与状态转移

接下来我们讨论如何将贡献在状态转移 $f_{i,j}$ 的状态中表示。按照 i 是否为不动点，分两种情况讨论：

若权值为 i 的人是动点，则贡献是它的实际位置加上以 j 为结尾的连续子段的起始位置（因为我们会将动点接在不动点的前面）的实际位置。依照我们上面的思路计算贡献。后者是简单的，因为后者的贡献没有受到不动点的影响（因为 j 是位置最前的不动点），只需要计算 j 前所有还未被考虑过的点的个数即可，写成公式是 $1 + \sum_{k < j} [p_k < i]$ 。前者则比较复杂，因为除了未被考虑过的点，它还受到位置在

其之前的不动点的影响。由于不动点的放置情况在状态中被抹去了，所以在此时计算这个贡献是困难的，我们不妨将它提前，在确定不动点的过程中确定这部分的贡献。

若权值为 i 的人是不动点，那么足以确定的是，假设 i 在原排列中的位置是 pos_i ，那么在区间 $[pos_i + 1, j - 1]$ 内的所有点都是动点，即它们都需要计算上面提到的不动点的贡献。对于 $k \in [pos_i + 1, j - 1]$ ，所有权值在区间 $[p_k + 1, i]$ 的人都在它前面的不动点的连续子段中出现过，可以得到这部分的贡献为 $\max\{0, i - p_k\}$ 。

综上，不难得到转移方程：

- $f_{i,j} \leftarrow f_{i+1,j} + c(k, i) + c(pos_i, i)$ ，其中 $c(x, y)$ 表示 $1 + \sum_{i < x} [p_i < y]$ 。
- $f_{i,pos_i} \leftarrow f_{i+1,j} + \sum_{pos_i < k < j} \max\{i - p_k, 0\}$ 。

直接转移即可，时间复杂度 $\mathcal{O}(n^2)$ 。